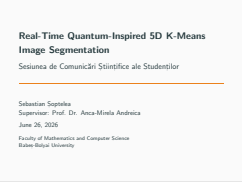
Real-Time Quantum-Inspired 5D K-Means Image Segmentation

Sesiunea de Comunicări Științifice ale Studenților

Sebastian Șoptelea
Supervisor: Prof. Dr. Anca-Mirela Andreica

June 26, 2026

Faculty of Mathematics and Computer Science
Babeș-Bolyai University



Hi my name is Sebastian Soptelea and I'm here to present my paper titled Real-time quantum inspired 5D kmeans image segmentation, realised under the supervision of prof. dr anca andreica

This project bridges high-performance GPU programming with quantum-inspired metrics to solve a critical real-time computer vision problem.

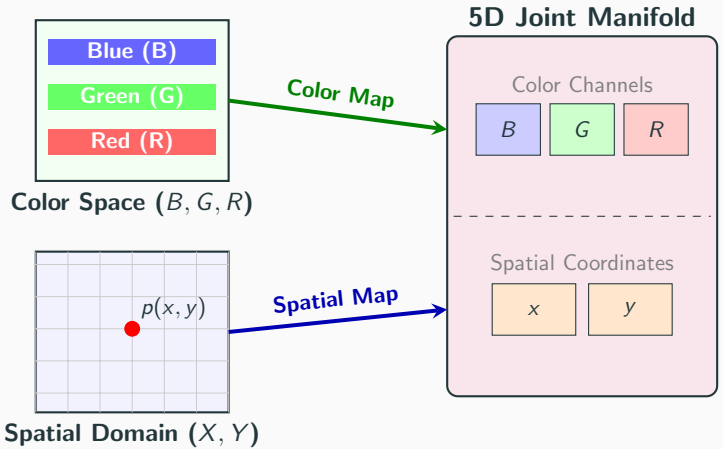
Problem Statement: The Challenge

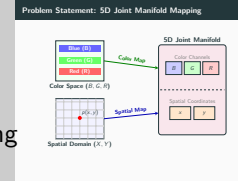
- **Core Goal:** Achieving high-fidelity joint color-spatial (5D) image segmentation under strict real-time constraints (> 120 FPS) for live video.
- **The Segmentation Problem:** Traditional 3D color-only segmentation (e.g., in HSV or BGR) lacks spatial context, leading to highly fragmented regions.
- **5D Joint Manifold (B, G, R, X, Y):** Grouping pixels by both color and coordinates preserves spatial consistency, requiring dimension normalization and hardware-level acceleration to handle the computational overhead.

Problem Statement: The Challenge
• Core Goal: Achieving high-fidelity joint color-spatial (5D) image segmentation under strict real-time constraints (> 120 FPS) for live video.
• The Segmentation Problem: Traditional 3D color-only segmentation (e.g., in HSV or BGR) lacks spatial context, leading to highly fragmented regions.
• 5D Joint Manifold (B, G, R, X, Y): Grouping pixels by both color and coordinates preserves spatial consistency, requiring dimension normalization and hardware-level acceleration to handle the computational overhead.

The core goal of this work is achieving high-fidelity joint color-spatial 5D segmentation under strict real-time constraints for live video.
Traditional 3D color-only methods (such as BGR/HSV) lack spatial context, which can lead to highly fragmented regions.
This is why we use a 5D joint manifold which, while it preserves spatial consistency, requires dimension normalization and hardware acceleration to handle the computational overhead.

Problem Statement: 5D Joint Manifold Mapping





To represent this visually, this diagram shows how the joint manifold is structured. For each pixel, we concatenate the three BGR color channels with their corresponding spatial X and Y coordinates to construct a single 5D vector.

Problem Statement: Requirements & Bottlenecks

- **Strict Real-Time Requirement (120+ FPS):** Processing budget of < 8.33 ms per frame for interactive computer vision (e.g., robotic loops, driving simulators).
- **Hardware Bottlenecks:**
  - *CPU Limits:* Sequential 5D clustering requires millions of calculations per frame (too slow for live feeds).
  - *NISQ QPU Limits:* Real Quantum Processing Units suffer from physical noise, state preparation overhead, and massive network latency.
- **Solution:** A parallel C++/CUDA pipeline that simulates quantum state overlaps directly on local classical GPU cores.

Problem Statement: Requirements & Bottlenecks
- **Strict Real-Time Requirement (120+ FPS):** Processing budget of < 8.33 ms per frame for interactive computer vision (e.g., robotic loops, driving simulators).
- **Hardware Bottlenecks:**
  - *CPU Limits:* Sequential 5D clustering requires millions of calculations per frame (too slow for live feeds).
  - *NISQ QPU Limits:* Real Quantum Processing Units suffer from physical noise, state preparation overhead, and massive network latency.
- **Solution:** A parallel C++/CUDA pipeline that simulates quantum state overlaps directly on local classical GPU cores.

To set the scene, high-speed interactive applications like robotics or driving simulators require a framerate of at least 120 frames per second to avoid latency, giving us a processing budget of under 8.3 milliseconds per frame. Meeting this constraint is challenging because clustering our 5D manifold introduces massive computational overhead, creating a severe bottleneck. Standard CPUs cannot handle these calculations in real time, and physical quantum processors are currently too noisy and slow. This is why we choose to simulate the quantum math locally on a GPU.

Original Contributions & What I Bring

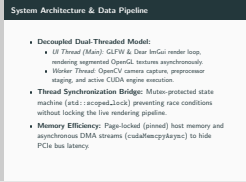
- **Custom High-Throughput Software Package:** Fully asynchronous CUDA pipeline decoupling the UI from the computational backend.
- **Hardware-Accelerated Quantum Cosine Engine:** Custom kernel utilizing SFU math intrinsics to emulate the SWAP test in real-time.
- **Asynchronous Still-Frame Diagnostics:** On-demand diagnostics on a background thread computing the 3 key quality metrics (WCSS, Davies-Bouldin Index, and Monte Carlo-approximated Silhouette Coefficient) without GUI lag.
- **Two-Tier Atomic Reduction Kernel:** Maximized memory throughput by aggregating sub-totals in block shared memory.

Original Contributions & What I Bring
- Custom High-Throughput Software Package: Fully asynchronous CUDA pipeline decoupling the UI from the computational backend.
- Hardware-Accelerated Quantum Cosine Engine: Custom kernel utilizing SFU math intrinsics to emulate the SWAP test in real-time.
- Asynchronous Still-Frame Diagnostics: On-demand diagnostics on a background thread computing the 3 key quality metrics (WCSS, Davies-Bouldin Index, and Monte Carlo-approximated Silhouette Coefficient) without GUI lag.
- Two-Tier Atomic Reduction Kernel: Maximized memory throughput by aggregating sub-totals in block shared memory.

To address these bottlenecks, my work introduces several original contributions. First, I built a fully asynchronous C++/CUDA pipeline that decouples the UI from the computational backend. Second, I developed a custom kernel that utilizes GPU Special Function Unit math intrinsics to emulate the quantum SWAP test in real time. Third, we implemented a non-blocking background thread to run on-demand diagnostics, computing quality metrics like WCSS and Silhouette without lag. Finally, we designed a two-tier atomic reduction kernel to maximize memory throughput.

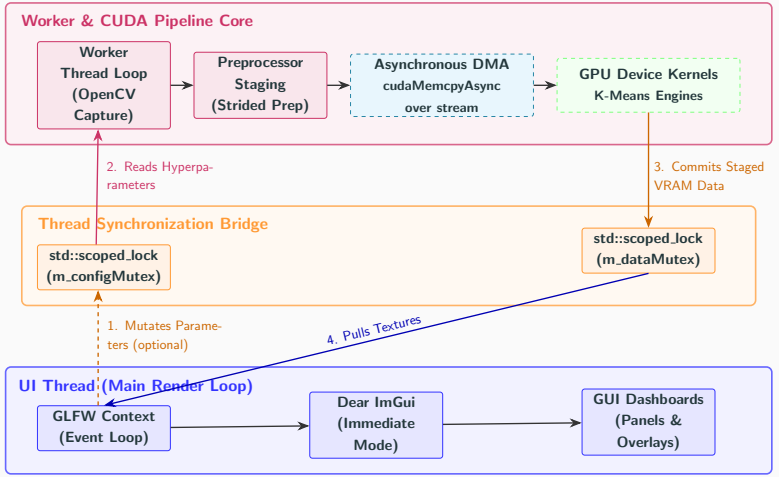
System Architecture & Data Pipeline

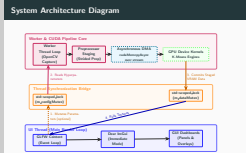
- **Decoupled Dual-Threaded Model:**
  - *UI Thread (Main):* GLFW & Dear ImGui render loop, rendering segmented OpenGL textures asynchronously.
  - *Worker Thread:* OpenCV camera capture, preprocessor staging, and active CUDA engine execution.
- **Thread Synchronization Bridge:** Mutex-protected state machine (`std::scoped_lock`) preventing race conditions without locking the live rendering pipeline.
- **Memory Efficiency:** Page-locked (pinned) host memory and asynchronous DMA streams (`cudaMemcpyAsync`) to hide PCIe bus latency.



To achieve the goals mentioned earlier, I developed a system architecture based on a decoupled, dual-threaded model. The main UI thread handles rendering the segmented frames, while a separate worker thread captures camera input and runs the CUDA engine. They communicate asynchronously via a mutex-protected thread synchronization bridge. To hide PCIe bus transfer latency, we employ page-locked host memory and non-blocking, asynchronous CUDA streams.

System Architecture Diagram

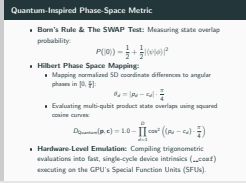




To understand how this architecture operates in practice, let's trace the lifecycle of a single video frame processing iteration. First, the UI thread optionally mutates hyperparameters like stride or the cluster count. These are passed through the thread synchronization bridge, where the worker thread reads them. Next, the worker thread handles frame preprocessing and dispatches pixel data to the GPU via asynchronous Direct memory access. Once the GPU completes the segmentation, the processed frame is committed back through the synchronization bridge to the main render loop to update the display.

Quantum-Inspired Phase-Space Metric

- **Born's Rule & The SWAP Test:** Measuring state overlap probability:
$$P(|0\rangle) = \frac{1}{2} + \frac{1}{2}|\langle\psi|\phi\rangle|^2$$
- **Hilbert Phase Space Mapping:**
  - Mapping normalized 5D coordinate differences to angular phases in $[0, \frac{\pi}{4}]$:
$$\theta_d = |p_d - c_d| \cdot \frac{\pi}{4}$$
  - Evaluating multi-qubit product state overlaps using squared cosine curves:
$$D_{\text{Quantum}}(\mathbf{p}, \mathbf{c}) = 1.0 - \prod_{d=1}^{D} \cos^2\left((p_d - c_d) \cdot \frac{\pi}{4}\right)$$
- **Hardware-Level Emulation:** Compiling trigonometric evaluations into fast, single-cycle device intrinsics (`__cosf`) executing on the GPU's Special Function Units (SFUs).



Having traced the pipeline's structure, let's examine the quantum-inspired phase-space metric, which is the alternative distance metric introduced in this work. Rather than using traditional Euclidean distance, we measure the overlap probability of two quantum states in Hilbert space, representing a simulation of the SWAP test. We map the normalized 5D coordinate differences to angular phases in the interval $[0, \pi/4]$ and compute their overlap using a product of squared cosines. Because this is computationally expensive, we compile these cosine calculations directly into single-cycle hardware intrinsics executing on the GPU's Special Function Units.

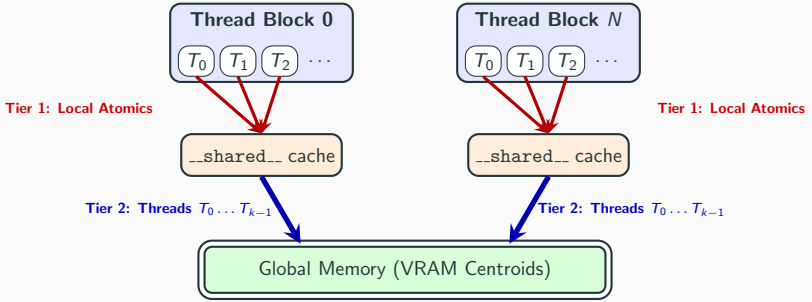
GPU Kernel & Memory Optimizations

- **GPU-Native Preprocessor:** Strided preprocessor formatting BGR-XY coordinates into an Array of Structures (AoS), maximizing L_1 cache line hit rates and supporting configurable stride-based spatial downsampling.
- **Templated Loop Unrolling:** Forcing compiler branch elimination via 5D templated math utilities (`#pragma unroll`) and fast math compiler flags.
- **Two-Tier Atomic Reduction:** A hierarchical aggregation strategy (detailed in the next slide) designed to minimize global memory traffic bottlenecks.

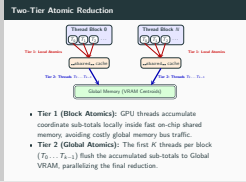
GPU Kernel & Memory Optimizations
- **GPU-Native Preprocessor:** Strided preprocessor formatting BGR-XY coordinates into an Array of Structures (AoS), maximizing L_1 cache line hit rates and supporting configurable stride-based spatial downsampling.
- **Templated Loop Unrolling:** Forcing compiler branch elimination via 5D templated math utilities (`#pragma unroll`) and fast math compiler flags.
- **Two-Tier Atomic Reduction:** A hierarchical aggregation strategy (detailed in the next slide) designed to minimize global memory traffic bottlenecks.

To run this quantum emulation in real time, I implemented several hardware-level GPU optimizations. First, the GPU-native preprocessor formats pixel data into an Array of Structures to maximize L1 cache hit rates, while also handling spatial downsampling using a configurable stride parameter. Second, we use templated vector math and compiler pragmas to force complete loop unrolling and eliminate branching. Finally, we address the global memory writeback bottleneck using a two-tier atomic reduction kernel, which we will detail in the next slide.

Two-Tier Atomic Reduction



- **Tier 1 (Block Atomics):** GPU threads accumulate coordinate sub-totals locally inside fast on-chip shared memory, avoiding costly global memory bus traffic.
- **Tier 2 (Global Atomics):** The first K threads per block ($T_0 \dots T_{k-1}$) flush the accumulated sub-totals to Global VRAM, parallelizing the final reduction.

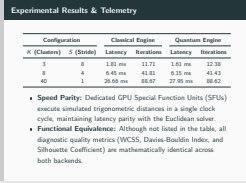


Looking at this two-tier atomic reduction diagram, we can see exactly how the memory bottleneck is resolved. In Tier 1, all threads accumulate cluster coordinate sums and pixel counts locally inside the fast, on-chip shared memory of their block using block atomics, which dramatically cuts down global bus traffic. Then, in Tier 2, only the first K threads per block—from T_0 to T_{k-1}—flush these pre-aggregated sums and counts to the global VRAM centroid arrays in parallel. Because the block size is 256, this two-tier structure reduces global memory contention by 256 times. These final values are then used to compute the updated cluster centroids, which ultimately determine which pixels get assigned to each cluster and how they are colored.

Experimental Results & Telemetry

Configuration		Classical Engine		Quantum Engine	
K (Clusters)	S (Stride)	Latency	Iterations	Latency	Iterations
3	8	1.81 ms	11.71	1.61 ms	12.38
8	4	6.45 ms	41.81	6.15 ms	41.43
40	1	26.66 ms	88.67	27.95 ms	88.62

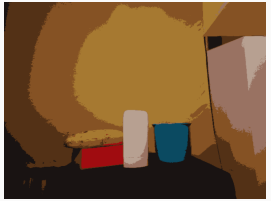
- **Speed Parity:** Dedicated GPU Special Function Units (SFUs) execute simulated trigonometric distances in a single clock cycle, maintaining latency parity with the Euclidean solver.
- **Functional Equivalence:** Although not listed in the table, all diagnostic quality metrics (WCSS, Davies-Bouldin Index, and Silhouette Coefficient) are mathematically identical across both backends.



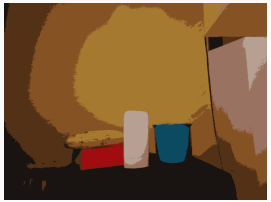
These hardware optimizations translate directly into the performance telemetry shown in this benchmark table. As you can see, the quantum-inspired engine maintains complete speed parity with the classical Euclidean baseline. For example, with $K = 3$ and a stride of 8, both run in under 1.8 milliseconds. This confirms that compiling the quantum metric's trigonometric evaluations into fast, single-cycle Special function unit instructions prevents any computational overhead. Furthermore, all quality metrics like within cluster sum of squares and Davies-Bouldin are mathematically identical across both solvers, confirming their functional equivalence.

Visual Segmentation Comparison

Original Image

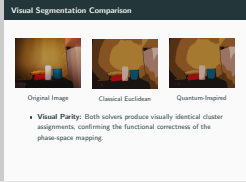


Classical Euclidean



Quantum-Inspired

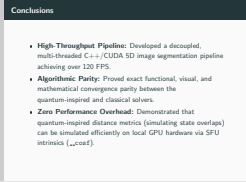
- **Visual Parity:** Both solvers produce visually identical cluster assignments, confirming the functional correctness of the phase-space mapping.



Here we can see a side-by-side visual comparison of the segmentation output. The original image on the left is segmented into identical spatial color clusters by both the classical Euclidean engine in the middle and the quantum-inspired engine on the right. Because the quantum phase distance is a monotonic mapping of feature similarity, it yields identical convergence states and final metrics. This demonstrates and validates the complete functional correctness of our quantum emulation.

Conclusions

- **High-Throughput Pipeline:** Developed a decoupled, multi-threaded C++/CUDA 5D image segmentation pipeline achieving over 120 FPS.
- **Algorithmic Parity:** Proved exact functional, visual, and mathematical convergence parity between the quantum-inspired and classical solvers.
- **Zero Performance Overhead:** Demonstrated that quantum-inspired distance metrics (simulating state overlaps) can be simulated efficiently on local GPU hardware via SFU intrinsics (`__cosf`).



To conclude, I would like to summarize our main findings. First, we successfully developed a decoupled, multi-threaded C++/CUDA pipeline that comfortably exceeds our 120 FPS real-time target. Second, we proved exact functional, visual, and mathematical convergence parity between the quantum-inspired solver and the classical baseline. Finally, we demonstrated that emulating quantum state overlaps in Hilbert space introduces zero performance overhead because these calculations are compiled directly into fast, single-cycle GPU hardware intrinsics on the Special Function Units.

Publications & Academic Activity

- **Book Chapter (HAIS 2026):**
  Şoptelea, S., Kallay, P., Tudor, M., Lung, R.I. (2027). *A Variational Quantum Classifier Applied to Romanian Financial Data*. In: Hybrid Artificial Intelligent Systems. Lecture Notes in Computer Science, vol 16596, pp. 429–440. Springer, Cham. DOI: 10.1007/978-3-032-29292-6_35.
- **Ongoing Research:** A comprehensive Quantum Machine Learning (QML) literature survey in the works.

Publications & Academic Activity

- **Book Chapter (HAIS 2026):**
  Şoptelea, S., Kallay, P., Tudor, M., Lung, R.I. (2027). *A Variational Quantum Classifier Applied to Romanian Financial Data*. In: Hybrid Artificial Intelligent Systems. Lecture Notes in Computer Science, vol 16596, pp. 429–440. Springer, Cham. DOI: 10.1007/978-3-032-29292-6_35.
- **Ongoing Research:** A comprehensive Quantum Machine Learning (QML) literature survey in the works.

In addition to the core findings of this thesis, I would like to highlight my related academic work. This research was inspired by my first-author book chapter published in Springer for HAIS 2026, which applied variational quantum classifiers to Romanian financial data. Building on these concepts, I also have an ongoing comprehensive Quantum Machine Learning literature survey in preparation.

Future Work

- **Spatiotemporal Tracking:** Integrating temporal continuity constraints (e.g., optical flow) to stabilize cluster boundaries across streaming video frames.
- **Physical NISQ QPU Deployment:** Migrating from local GPU-simulated phase overlaps to physical QPUs (e.g., via CUDA-Q) to measure true quantum state overlap latency.

Future Work
- **Spatiotemporal Tracking:** Integrating temporal continuity constraints (e.g., optical flow) to stabilize cluster boundaries across streaming video frames.
- **Physical NISQ QPU Deployment:** Migrating from local GPU-simulated phase overlaps to physical QPUs (e.g., via CUDA-Q) to measure true quantum state overlap latency

Looking ahead, there are two key avenues for expanding this research. First, we plan to integrate temporal continuity constraints, such as optical flow, to improve stability and prevent cluster boundaries from jittering across consecutive video frames. Second, we aim to transition from GPU emulation to physical NISQ-era quantum hardware using frameworks like CUDA-Q, allowing us to measure true quantum state overlap latency on actual QPUs.

Thank You!

Questions & Answers

With that, I would like to thank the committee and everyone in the public for your time and attention. I am now happy to open the floor to any questions you may have.